```
clc;
clear;
close all;
tic; % Start execution timer

%%
%% PART 1: VEHICLE, DRIVETRAIN & BATTERY PARAMETERS (ORIGINAL MODEL)
%%
% Vehicle Dynamics Parameters
m = 1800;               % Vehicle mass (kg)
g = 9.81;               % Gravity (m/s^2)
Crr = 0.01;             % Rolling resistance coefficient
rho = 1.225;            % Air density (kg/m^3)
Cd = 0.29;              % Drag coefficient
A = 2.2;                % Frontal area (m^2)
delta = 0.08;           % Rotational mass factor
m_eq = m * (1 + delta);  % Equivalent inertial mass (kg)
r_wheel = 0.3;          % Wheel radius (m)
gear_ratio = 9;         % Transmission gear ratio
eta_tire = 0.95;        % Tire mechanical efficiency
eta_drivetrain = 0.95;   % Drivetrain efficiency

% Motor & Inverter Parameters
T_motor_max = 250;       % Max motoring torque (Nm)
T_regen_max = 120;       % Max regenerative torque (Nm)
P_motor_max = 80e3;      % Max motor mechanical power (W)
v_min_regen = 1.5;       % Minimum velocity for regen braking (m/s)

% Battery Pack Physical Parameters (Restored to Baseline Design)
Batt_capacity = 50;      % Battery capacity (kWh)
V_nom = 350;             % Nominal pack voltage (V)
Batt_Cap_Ah = (Batt_capacity * 1000) / V_nom; % Ah Capacity (~142.86 Ah)
SOC_init = 0.90;         % Initial SOC (90%)
dU_dT = -0.00022;        % Entropic coefficient (V/K) [Reversible Heat]
C_th = 45000;            % Thermal capacitance (J/K)
R_th = 1.5;              % Passive thermal resistance to ambient (K/W)
P_accessory = 900;       % Continuous accessory electrical power (W)

%%
```

```
=
40 %% PART 2: MULTI-PHASE MISSION PROFILE GENERATION (2000s Total)
41 %%↙
==========================================================================↙
=
42 dt = 1;                % Simulation timestep (s)
43 t = (0:dt:2000)';       % Master time vector (0 to 2000 seconds)
44 N = length(t);
45
46 v = zeros(N, 1);             % Vehicle speed (m/s)
47 T_amb = zeros(N, 1);         % Ambient temperature (°C)
48 I_charge_demand = zeros(N, 1); % External fast charge current profile (A)
49
50 % --- Scenario 3: Dynamic Ambient Temperature Ramp ---
51 % Ambient temperature ramps from 25°C to 38°C (Extreme Summer Conditions)
52 T_amb = 25 + 13 * (1 - exp(-t / 1200));
53
54 % --- Scenario 1: Aggressive Driving / Rapid Accel & Decel (0s to 600s) ---
55 t_drive = t(1:601);
56 v_drive = 22 * sin(2*pi*t_drive/120) + 12 * sin(2*pi*t_drive/30) + 15;
57 v_drive(v_drive < 0) = 0;
58 v(1:601) = v_drive;
59
60 % --- Transition Period (601s to 700s) ---
61 v(602:701) = 0; % Vehicle comes to rest at fast-charging station
62
63 % --- Scenario 2: High-Power DC Fast Charging (701s to 1600s) ---
64 % 250 kW High-Current surge (300A CC-CV fast charging sequence)
65 t_charge = t(702:1601) - t(701);
66 I_charge_profile = -300 * exp(-t_charge / 600); % Negative sign = Charging current
67 I_charge_profile(I_charge_profile > -100) = -100; % Charge floor
68 I_charge_demand(702:1601) = I_charge_profile;
69
70 % --- Post-Charge Soak / Idle Period (1601s to 2000s) ---
71 v(1602:end) = 0;
72
73 %%↙
==========================================================================↙
=
74 %% PART 3: LONGITUDINAL VEHICLE DYNAMICS & POWER MAPPING
75 %%↙
==========================================================================↙
=
76 % Acceleration vector calculation
```

```
77 a = [diff(v)/dt; 0];
78 a_max = 3.0; a_min = -4.0;
79 a(a > a_max) = a_max;
80 a(a < a_min) = a_min;
81
82 % Throttle Position Calculation (%) for Telematics Lookahead
83 theta_throttle = min(100, max(0, (a / a_max) * 100));
84
85 % Wheel rotational speed
86 omega_wheel = v / r_wheel;
87 omega = omega_wheel * gear_ratio;
88 omega_base = P_motor_max / T_motor_max;
89 omega_max = max(omega) + eps;
90
91 % Force calculations
92 F_rr = m * g * Crr;
93 F_aero = 0.5 * rho * Cd * A .* v.^2;
94 F_acc = m_eq .* a;
95 F_total = F_rr + F_aero + F_acc;
96
97 % Mechanical wheel power
98 P_wheel = (F_total .* v) / eta_tire;
99
100 % Motor torque limits and mechanical power flow
101 P_mech = zeros(N, 1);
102 for i = 1:N
103     if P_wheel(i) > 0
104         if omega(i) <= omega_base
105             T_avail = T_motor_max;
106         else
107             T_avail = P_motor_max / omega(i);
108         end
109         P_avail = T_avail * omega(i);
110         P_mech(i) = min(P_wheel(i), P_avail);
111     elseif P_wheel(i) < 0 && v(i) > v_min_regen
112         P_regen_limit = T_regen_max * omega(i);
113         P_mech(i) = -min(abs(P_wheel(i)), P_regen_limit);
114     else
115         P_mech(i) = 0;
116     end
117 end
118 P_mech = P_mech / eta_drivetrain;
119
120 % Motor and inverter efficiency mapping
```

```
121 drive_eff = ones(N, 1);
122 regen_eff = ones(N, 1);
123 for i = 1:N
124     if P_mech(i) > 0
125         T_req = P_mech(i) / max(omega(i), eps);
126         Tn = abs(T_req) / T_motor_max;
127         on = omega(i) / omega_max;
128         drive_eff(i) = 0.90 - 0.15*(1-on)^2 - 0.20*(Tn-0.6)^2;
129         drive_eff(i) = min(max(drive_eff(i), 0.75), 0.92);
130     elseif P_mech(i) < 0
131         T_req = abs(P_mech(i)) / max(omega(i), eps);
132         Tn = T_req / T_regen_max;
133         on = omega(i) / omega_max;
134         regen_eff(i) = 0.75 - 0.20*(1-on)^2 - 0.30*(Tn-0.5)^2;
135         regen_eff(i) = min(max(regen_eff(i), 0.55), 0.80);
136     end
137 end
138
139 % Electrical power calculations
140 P_batt_disch_max = 90e3;
141 P_batt_char_max  = 40e3;
142 P_drive = zeros(N, 1);
143 P_regen = zeros(N, 1);
144
145 for i = 1:N
146     if P_mech(i) > 0
147         P_drive(i) = P_mech(i) / drive_eff(i);
148         P_drive(i) = min(P_drive(i), P_batt_disch_max);
149     elseif P_mech(i) < 0
150         P_regen(i) = abs(P_mech(i)) * regen_eff(i);
151         P_regen(i) = min(P_regen(i), P_batt_char_max);
152     end
153 end
154
155 % Total battery power required during drive cycle
156 P_batt_drive = P_drive - P_regen + (v > 0) * P_accessory;
157
158 %%↙
================================================================↙
=
159 %% PART 4: 2D INTERNAL RESISTANCE & ELECTRICAL LOOKUP DEFINITION
160 %%↙
================================================================↙
=
```

```
161 SOC_curve = [0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0];
162 OCV_curve = [300, 320, 335, 345, 350, 355, 360, 365, 372, 380, 390]; % OCV (V)
163
164 % 2D Lookup Table for Internal Resistance R_int(SOC, T) in Ohms
165 T_vec   = [0,   15,   25,   40,   55];     % Temperature grid (°C)
166 SOC_vec = [0.1, 0.2,  0.5,  0.8,  1.0];    % SOC grid
167
168 R_i_map = [
169     0.150, 0.080, 0.060, 0.045, 0.042; % SOC 10%
170     0.130, 0.070, 0.052, 0.040, 0.038; % SOC 20%
171     0.120, 0.065, 0.048, 0.035, 0.034; % SOC 50%
172     0.125, 0.068, 0.050, 0.037, 0.035; % SOC 80%
173     0.140, 0.075, 0.055, 0.042, 0.040  % SOC 100%
174 ];
175
176 % Interpolant Object
177 F_Rint = griddedInterpolant({SOC_vec, T_vec}, R_i_map, 'linear', 'linear');
178
179 %%↙
================================================================↙
=
180 %% PART 5: COULOMB COUNTING & HIGH-FIDELITY HEAT GENERATION LOOP
181 %%↙
================================================================↙
=
182 I_batt = zeros(N, 1);
183 SOC = zeros(N, 1);
184 T_batt = zeros(N, 1);
185 V_oc_vec = zeros(N, 1);
186 V_batt = zeros(N, 1);
187 R_internal_vec = zeros(N, 1);
188
189 Q_ohmic = zeros(N, 1);
190 Q_entropic = zeros(N, 1);
191 Q_heat = zeros(N, 1);
192
193 % Initial conditions
194 SOC(1) = SOC_init;
195 T_batt(1) = T_amb(1);
196
197
198 % Active HVAC & Cooling Circuit Parameters
199 COP_chiller = 2.5;       % Coefficient of Performance for chiller
200 P_pump = 50;             % Coolant pump electrical power consumption (W)
```

```
201 s_charger = 8820;        % DC Fast Charger location along route (meters)
202
203 for i = 1:N
204     % Current calculation: sum of traction load and fast charge demand
205     if I_charge_demand(i) < 0
206         I_batt(i) = I_charge_demand(i); % DC Fast Charge phase
207     else
208         I_batt(i) = P_batt_drive(i) / V_nom; % Traction / Regen phase
209     end
210
211     % Coulomb counting SOC update
212     if i > 1
213         E_step_Wh = (I_batt(i) * V_nom * dt) / 3600;
214         SOC(i) = SOC(i-1) - E_step_Wh / (Batt_capacity * 1000);
215         SOC(i) = max(0.05, min(1.0, SOC(i)));
216     end
217
218     % OCV Lookup
219     V_oc_vec(i) = interp1(SOC_curve, OCV_curve, SOC(i), 'linear', 'extrap');
220
221     % 2D Lookup for internal resistance R_int(SOC, T_batt)
222     % CORRECTED CODE FOR PART 5
223     T_curr = T_batt(i);
224     SOC_curr = SOC(i);
225     R_internal_vec(i) = F_Rint(SOC_curr, T_curr);
226
227     % Thermodynamic Heat Formulation:
228     % 1. Ohmic (Irreversible) loss: Q_ohmic = I^2 * R
229     Q_ohmic(i) = (I_batt(i)^2) * R_internal_vec(i);
230
231     % 2. Entropic (Reversible) loss: Q_entropic = I * T_kelvin * (dU/dT)
232     T_kelvin = T_curr + 273.15;
233     Q_entropic(i) = I_batt(i) * T_kelvin * dU_dT;
234
235     % 3. Total Heat Generation
236     Q_heat(i) = Q_ohmic(i) + Q_entropic(i);
237
238     % Terminal Voltage Calculation
239     V_batt(i) = V_oc_vec(i) - I_batt(i) * R_internal_vec(i);
240
241     % Uncooled open-loop baseline thermal update
242     if i < N
243         Q_passive_cooling = (T_curr - T_amb(i)) / R_th;
244         dT = (Q_heat(i) - Q_passive_cooling) / C_th;
```

```
245         T_batt(i+1) = T_curr + dT * dt;
246     end
247 end
248
249 %%↙
==========================================================================↙
=
250 %% PART 5B: STEP 4 - BASELINE ACTIVE THERMAL CONTROLLER (HYSTERESIS)
251 %%↙
==========================================================================↙
=
252 % Controller Settings
253 T_on  = 35.0;     % Turn-ON threshold (°C)
254 T_off = 32.0;     % Turn-OFF threshold (°C)
255 Q_cooling_max = 2000; % Active cooling power capacity (W)
256
257 % Tracking Vectors
258 T_batt_baseline      = zeros(N, 1);
259 Q_cooling_baseline   = zeros(N, 1);
260 cooling_active_state = zeros(N, 1); % Binary state: 1 = ON, 0 = OFF
261
262 % Initial Condition
263 T_batt_baseline(1) = T_amb(1);
264 is_cooling_on = false;
265
266 % Baseline Thermal Simulation Loop
267 for i = 1:N
268     T_curr = T_batt_baseline(i);
269
270     % Hysteresis Controller Logic
271     if T_curr >= T_on
272         is_cooling_on = true;
273     elseif T_curr <= T_off
274         is_cooling_on = false;
275     end
276
277     % Apply Active Cooling Power
278     if is_cooling_on
279         Q_cooling_baseline(i) = Q_cooling_max;
280         cooling_active_state(i) = 1;
281     else
282         Q_cooling_baseline(i) = 0;
283         cooling_active_state(i) = 0;
284     end
```

```
285
286     % Dynamic Thermal Update (Heat Gen - Passive Heat Loss - Active Cooling)
287     if i < N
288         Q_passive = (T_curr - T_amb(i)) / R_th;
289         dT = (Q_heat(i) - Q_passive - Q_cooling_baseline(i)) / C_th;
290         T_batt_baseline(i+1) = T_curr + dT * dt;
291     end
292 end
293
294 % Energy Metrics for Baseline Controller
295 % Energy Metrics for Baseline Controller (accounting for Chiller COP and Pump Power)
296 E_cooling_baseline_Wh = trapz(t, (Q_cooling_baseline / COP_chiller) +↙
(Q_cooling_baseline > 0) * P_pump) / 3600;
297
298 % Print Baseline Controller Performance
299 fprintf('\n============================================================\n');
300 fprintf('     STEP 4: BASELINE CONTROLLER PERFORMANCE SUMMARY     \n');
301 fprintf('============================================================\n');
302 fprintf('Uncooled Max Temperature    : %.2f °C\n', max(T_batt));
303 fprintf('Baseline Max Temperature    : %.2f °C\n', max(T_batt_baseline));
304 fprintf('Total Active Cooling Energy : %.2f Wh\n', E_cooling_baseline_Wh);
305 fprintf('Total Cooling Runtime       : %.0f seconds\n', sum(cooling_active_state) * dt);
306 fprintf('============================================================\n\n');
307
308 %% Visualizing Step 4 Performance
309 figure('Name', 'Step 4: Baseline Active Cooling Performance', 'NumberTitle', 'off');
310 subplot(2,1,1);
311 plot(t, T_batt, 'r--', 'LineWidth', 1.5); hold on;
312 plot(t, T_batt_baseline, 'b', 'LineWidth', 2);
313 yline(T_on, 'k--', 'Turn-ON (35°C)');
314 yline(T_off, 'g--', 'Turn-OFF (32°C)');
315 ylabel('Temp [°C]'); grid on;
316 legend('Uncooled Baseline', 'Step 4 Reactive Baseline', 'Location', 'northwest');
317 title('Battery Temperature: Uncooled vs. Step 4 Active Controller');
318
319 subplot(2,1,2);
320 plot(t, Q_cooling_baseline, 'm', 'LineWidth', 1.5);
321 ylabel('Cooling Power [W]'); xlabel('Time [s]'); grid on;
322 title('Step 4 Baseline Active Cooling Power Applied');
323
324 %%↙
============================================================↙
=
325 %% PART 5C: STEP 5 - PREDICTIVE THERMAL CONTROLLER (LOOKAHEAD PRE-↙
```

```
COOLING)
%%↙
==========================================================================↙
=
% Predictive Controller Hyperparameters
H_p = 120;                  % Lookahead prediction horizon (seconds)
Q_thresh = 1200;            % Predicted heat generation threshold (W)
T_pre_target = 28.0;        % Pre-cooling lower limit target (°C)
T_set_predictive = 32.0;    % Target controlled temperature (°C)
tau_control = 30.0;         % Proportional control time constant (s)

% Tracking Vectors
T_batt_predictive     = zeros(N, 1);
Q_cooling_predictive  = zeros(N, 1);
Q_pred_future         = zeros(N, 1);

% Initial Condition
T_batt_predictive(1) = T_amb(1);

% Predictive Simulation Loop
% Track Route Position for Telematics
s_route = cumtrapz(t, v);

% Predictive Simulation Loop with Telematics & Lookahead
for i = 1:N
    T_curr = T_batt_predictive(i);
    v_curr = v(i);

    % 1. Telematics: Charger Proximity & Time-to-Arrival
    d_to_charger = s_charger - s_route(i);
    if v_curr > 0.5
        t_to_charger = d_to_charger / v_curr;
    else
        t_to_charger = d_to_charger / max(1, mean(v));
    end
    incoming_charge_flag = (d_to_charger > 0) && (t_to_charger <= H_p);

    % 2. Telematics: Throttle Lookahead (15-second peak horizon)
    idx_throttle_end = min(N, i + 15);
    peak_throttle_lookahead = max(theta_throttle(i:idx_throttle_end));

    % 3. Lookahead Heat Generation
    idx_end = min(N, i + H_p);
    Q_pred_future(i) = mean(Q_heat(i:idx_end));
```

```
367
368     % --- DECISION LOGIC INCORPORATING TELEMATICS ---
369     if incoming_charge_flag && T_curr > T_pre_target
370         % Pre-cooling triggered by Location & Fast-Charge proximity
371         Q_cooling_predictive(i) = Q_cooling_max;
372
373     elseif peak_throttle_lookahead > 50 && T_curr > 29.0
374         % Pre-cooling triggered by Throttle demand spike
375         Q_cooling_predictive(i) = 0.75 * Q_cooling_max;
376
377     elseif T_curr >= T_set_predictive
378         % Proportional active setpoint maintenance
379         P_term = C_th * (T_curr - T_set_predictive) / tau_control;
380         Q_req = Q_pred_future(i) + P_term;
381         Q_cooling_predictive(i) = min(Q_cooling_max, max(0, Q_req));
382
383     else
384         Q_cooling_predictive(i) = 0;
385     end
386
387     % Dynamic Thermal State Integration
388     if i < N
389         Q_passive = (T_curr - T_amb(i)) / R_th;
390         dT = (Q_heat(i) - Q_passive - Q_cooling_predictive(i)) / C_th;
391         T_batt_predictive(i+1) = T_curr + dT * dt;
392     end
393 end
394
395 % Energy Metrics Calculation (accounting for Chiller COP and Pump Power)
396 E_cooling_pred_Wh = trapz(t, (Q_cooling_predictive / COP_chiller) + (Q_cooling_predictive↙
> 0) * P_pump) / 3600;
397 E_savings_percent = ((E_cooling_baseline_Wh - E_cooling_pred_Wh) /↙
E_cooling_baseline_Wh) * 100;
398
399 % Print Step 5 Performance Summary
400 fprintf('\n============================================================\n');
401 fprintf('     STEP 5: PREDICTIVE CONTROLLER PERFORMANCE SUMMARY  \n');
402 fprintf('============================================================\n');
403 fprintf('Uncooled Max Temperature       : %.2f °C\n', max(T_batt));
404 fprintf('Step 4 Baseline Max Temp       : %.2f °C\n', max(T_batt_baseline));
405 fprintf('Step 5 Predictive Max Temp     : %.2f °C\n', max(T_batt_predictive));
406 fprintf('Baseline Active Energy         : %.2f Wh\n', E_cooling_baseline_Wh);
407 fprintf('Predictive Active Energy       : %.2f Wh\n', E_cooling_pred_Wh);
408 fprintf('Energy Savings vs. Baseline    : %.2f %%\n', E_savings_percent);
```

```
409 fprintf('============================================================\n\n');
410
411 %% Visualizing Step 4 vs. Step 5 Comparison
412 figure('Name', 'Step 5: Predictive vs. Reactive Thermal Control', 'NumberTitle', 'off');
413 subplot(2,1,1);
414 plot(t, T_batt, 'r--', 'LineWidth', 1.2); hold on;
415 plot(t, T_batt_baseline, 'b', 'LineWidth', 1.5);
416 plot(t, T_batt_predictive, 'g', 'LineWidth', 2);
417 yline(35.0, 'k--', 'Safety Limit (35°C)');
418 yline(32.0, 'm--', 'Target Operating Setpoint (32°C)');
419 ylabel('Temp [°C]'); grid on;
420 legend('Uncooled Baseline', 'Step 4 Reactive Hysteresis', 'Step 5 Predictive Lookahead',↙
'Location', 'northwest');
421 title('Battery Pack Temperature Response: Step 4 (Reactive) vs. Step 5 (Predictive)');
422
423 subplot(2,1,2);
424 plot(t, Q_cooling_baseline, 'b', 'LineWidth', 1.2); hold on;
425 plot(t, Q_cooling_predictive, 'g', 'LineWidth', 1.5);
426 ylabel('Cooling Power [W]'); xlabel('Time [s]'); grid on;
427 legend('Step 4 Reactive Power', 'Step 5 Predictive Power');
428 title('Active Cooling Power Demand Comparison');
429
430 %%↙
==========================================================================↙
=
431 %% PART 5D: STEP 5 - FULL CONSTRAINED MODEL PREDICTIVE CONTROLLER↙
(MPC)
432 %%↙
==========================================================================↙
=
433 % MPC Hyperparameters
434 H_p = 120;              % Prediction horizon (seconds)
435 T_set = 32.0;           % Target operational setpoint (°C)
436 T_max_limit = 35.0;       % Hard maximum temperature safety constraint (°C)
437 w_T = 100.0;            % Thermal error penalty weight
438 w_E = 1e-5;             % Energy consumption penalty weight
439 w_dU = 1e-3;            % Actuator slew rate penalty weight
440
441 % Tracking Vectors
442 T_batt_MPC = zeros(N, 1);
443 Q_cooling_MPC = zeros(N, 1);
444 T_batt_MPC(1) = T_amb(1);
445
446 % Optimization Options (Fast execution using interior-point / sqp)
```

```
447 opts = optimoptions('fmincon', 'Display', 'off', 'Algorithm', 'sqp', ...
448                     'MaxIterations', 30, 'OptimalityTolerance', 1e-3);
449
450 % Pre-allocate Decision Variable Initial Guess
451 u_init = zeros(H_p, 1);
452
453 fprintf('Running Full Receding-Horizon MPC Simulation over %d steps...\n', N);
454
455 % Receding-Horizon Control Loop
456 for i = 1:N
457     % Define Current State and Disturbance Vectors across Horizon
458     T_curr = T_batt_MPC(i);
459     idx_end = min(N, i + H_p - 1);
460     horizon_len = idx_end - i + 1;
461
462     Q_dist = Q_heat(i:idx_end);
463     T_a_dist = T_amb(i:idx_end);
464
465     % Zero-pad disturbance vectors if near the end of simulation
466     if horizon_len < H_p
467         Q_dist = [Q_dist; repmat(Q_dist(end), H_p - horizon_len, 1)];
468         T_a_dist = [T_a_dist; repmat(T_a_dist(end), H_p - horizon_len, 1)];
469     end
470
471     % Last applied control action for slew rate calculation
472     if i == 1
473         u_prev = 0;
474     else
475         u_prev = Q_cooling_MPC(i-1);
476     end
477
478     % Cost Handle and Strict Nonlinear Constraint
479     cost_fun    = @(u) mpc_cost_function(u, T_curr, Q_dist, T_a_dist, u_prev, ...
480                                          C_th, R_th, dt, H_p, T_set, w_T, w_E, w_dU);
481     nonlcon_fun = @(u) mpc_nonlcon(u, T_curr, Q_dist, T_a_dist, C_th, R_th, dt, H_p,↙
T_max_limit);
482
483     lb = zeros(H_p, 1);
484     ub = Q_cooling_max * ones(H_p, 1);
485
486     % Solve with nonlcon passed as 9th argument
487     [u_opt, ~] = fmincon(cost_fun, u_init, [], [], [], [], lb, ub, nonlcon_fun, opts);
488
489     % Receding Horizon Execution: Apply First Control Action
```

```
490     Q_cooling_MPC(i) = u_opt(1);
491
492     % Shift initial guess for next step (Warm Start)
493     u_init = [u_opt(2:end); u_opt(end)];
494
495     % Dynamic Thermal Integration of Plant
496     if i < N
497         Q_passive = (T_curr - T_amb(i)) / R_th;
498         dT = (Q_heat(i) - Q_passive - Q_cooling_MPC(i)) / C_th;
499         T_batt_MPC(i+1) = T_curr + dT * dt;
500     end
501 end
502
503 % Compute Final Metrics (accounting for Chiller COP and Pump Power)
504 E_cooling_MPC_Wh = trapz(t, (Q_cooling_MPC / COP_chiller) + (Q_cooling_MPC > 0) * ↙
P_pump) / 3600;
505 MPC_energy_savings = ((E_cooling_baseline_Wh - E_cooling_MPC_Wh) / ↙
E_cooling_baseline_Wh) * 100;
506
507 % Print Fully Completed Performance Results
508 fprintf('\n============================================================\n');
509 fprintf('     STEP 5: FULL MPC PERFORMANCE SUMMARY (COMPLETED)   \n');
510 fprintf('============================================================\n');
511 fprintf('Uncooled Peak Temperature      : %.2f °C\n', max(T_batt));
512 fprintf('Step 4 Baseline Max Temp       : %.2f °C\n', max(T_batt_baseline));
513 fprintf('Step 5 Full MPC Max Temp       : %.2f °C\n', max(T_batt_MPC));
514 fprintf('Baseline Active Energy         : %.2f Wh\n', E_cooling_baseline_Wh);
515 fprintf('Full MPC Active Energy         : %.2f Wh\n', E_cooling_MPC_Wh);
516 fprintf('Energy Savings vs. Baseline    : %.2f %%\n', MPC_energy_savings);
517 fprintf('============================================================\n\n');
518
519
520
521
522 %% Visualizing Full Step 5 MPC vs. Predictive vs. Baseline
523 figure('Name', 'Step 5: Full MPC Thermal Performance Comparison', 'NumberTitle', 'off');
524 subplot(2,1,1);
525 plot(t, T_batt, 'r--', 'LineWidth', 1.2); hold on;
526 plot(t, T_batt_baseline, 'b:', 'LineWidth', 1.5);
527 plot(t, T_batt_predictive, 'g--', 'LineWidth', 1.5);
528 plot(t, T_batt_MPC, 'm', 'LineWidth', 2);
529 yline(T_max_limit, 'k--', 'Safety Threshold (35°C)');
530 yline(T_set, 'c--', 'Target Setpoint (32°C)');
531 ylabel('Temp [°C]'); grid on;
```

```
532 legend('Uncooled', 'Step 4 Baseline (Hysteresis)', 'Step 5 Predictive', 'Step 5 Full MPC',↙
'Location', 'northwest');
533 title('Battery Pack Temperature Control: Step 4 vs. Step 5 (Predictive & MPC)');
534
535 subplot(2,1,2);
536 plot(t, Q_cooling_baseline, 'b:', 'LineWidth', 1.2); hold on; % Corrected comma here
537 plot(t, Q_cooling_predictive, 'g--', 'LineWidth', 1.2);
538 plot(t, Q_cooling_MPC, 'm', 'LineWidth', 1.5);
539 ylabel('Cooling Power [W]'); xlabel('Time [s]'); grid on;
540 legend('Baseline Power', 'Predictive Power', 'MPC Power');
541 title('Active Cooling Power Applied Across Controller Architectures');
542
543
544
545 %%↙
==========================================================================↙
=
546 %% PART 5E: STEP 6 - COMPREHENSIVE CONTROLLER BENCHMARKING &↙
ENERGY EVALUATION
547 %%↙
==========================================================================↙
=
548 fprintf('\nComputing Step 6 Comparative Benchmarks & Advanced Degradation↙
Metrics...\n');
549
550 % ------------------------------------------------------------------------
551 % 1. THERMAL PERFORMANCE & LIMIT VIOLATION METRICS
552 % ------------------------------------------------------------------------
553 T_peak_uncooled = max(T_batt);
554 T_peak_base     = max(T_batt_baseline);
555 T_peak_pred     = max(T_batt_predictive);
556 T_peak_mpc      = max(T_batt_MPC);
557
558 % Time spent above safety limit (35°C) and target setpoint (32°C) in seconds
559 time_above_35_uncooled = sum(T_batt > 35.0) * dt;
560 time_above_35_base     = sum(T_batt_baseline > 35.0) * dt;
561 time_above_35_pred     = sum(T_batt_predictive > 35.0) * dt;
562 time_above_35_mpc      = sum(T_batt_MPC > 35.0) * dt;
563
564 time_above_32_base     = sum(T_batt_baseline > 32.0) * dt;
565 time_above_32_pred     = sum(T_batt_predictive > 32.0) * dt;
566 time_above_32_mpc      = sum(T_batt_MPC > 32.0) * dt;
567
568 % ------------------------------------------------------------------------
```

```
569 % 2. ENERGY CONSUMPTION & SAVINGS CALCULATIONS
570 % ---------------------------------------------------------------------
571 % Energy values already computed in earlier parts (Wh)
572 E_base_Wh = E_cooling_baseline_Wh;
573 E_pred_Wh = E_cooling_pred_Wh;
574 E_mpc_Wh  = E_cooling_MPC_Wh;
575
576 % Relative Energy Savings vs Step 4 Reactive Baseline (%)
577 savings_pred_pct = ((E_base_Wh - E_pred_Wh) / E_base_Wh) * 100;
578 savings_mpc_pct  = ((E_base_Wh - E_mpc_Wh) / E_base_Wh) * 100;
579
580 % Cumulative Chiller Electrical Energy Trajectories over time (Wh)
581 E_cum_base = cumtrapz(t, (Q_cooling_baseline / COP_chiller) + (Q_cooling_baseline > 0) * ↙
P_pump) / 3600;
582 E_cum_pred = cumtrapz(t, (Q_cooling_predictive / COP_chiller) + (Q_cooling_predictive >↙
0) * P_pump) / 3600;
583 E_cum_mpc  = cumtrapz(t, (Q_cooling_MPC / COP_chiller) + (Q_cooling_MPC > 0) * ↙
P_pump) / 3600;
584
585 % ---------------------------------------------------------------------
586 % 3. ACTUATOR WEAR & SMOOTHNESS INDEX (Mean |dQ/dt|)
587 % ---------------------------------------------------------------------
588 wear_base = mean(abs(diff(Q_cooling_baseline))) / dt;
589 wear_pred = mean(abs(diff(Q_cooling_predictive))) / dt;
590 wear_mpc  = mean(abs(diff(Q_cooling_MPC))) / dt;
591
592 % ---------------------------------------------------------------------
593 % 4. ADVANCED WORK: BATTERY DEGRADATION & SOH IMPACT (Arrhenius↙
Model)
594 % ---------------------------------------------------------------------
595 % Empirical SEI Growth Kinetic parameters
596 E_a = 31700;    % Activation energy (J/mol)
597 R_gas = 8.314;  % Universal gas constant (J/mol*K)
598 A_pre = 1000;   % Pre-exponential scaling constant
599
600 % Instantaneous capacity degradation rate k_deg(T)
601 k_deg_uncooled = A_pre * exp(-E_a ./ (R_gas * (T_batt + 273.15)));
602 k_deg_base     = A_pre * exp(-E_a ./ (R_gas * (T_batt_baseline + 273.15)));
603 k_deg_pred     = A_pre * exp(-E_a ./ (R_gas * (T_batt_predictive + 273.15)));
604 k_deg_mpc      = A_pre * exp(-E_a ./ (R_gas * (T_batt_MPC + 273.15)));
605
606 % Cumulative aging factor relative to 25°C reference
607 aging_uncooled = trapz(t, k_deg_uncooled);
608 aging_base     = trapz(t, k_deg_base);
```

```
609 aging_pred     = trapz(t, k_deg_pred);
610 aging_mpc      = trapz(t, k_deg_mpc);
611
612 % Relative life extension vs Reactive Baseline (%)
613 life_ext_pred_pct = ((aging_base - aging_pred) / aging_base) * 100;
614 life_ext_mpc_pct  = ((aging_base - aging_mpc) / aging_base) * 100;
615
616 % Estimated full-equivalent battery cycle life extension
617 est_cycles_base = 1500; % Baseline reference cycles under thermal stress
618 est_cycles_mpc  = est_cycles_base * (aging_base / aging_mpc);
619
620 % ------------------------------------------------------------------------
621 % 5. ADVANCED WORK: VEHICLE RANGE EXTENSION IMPACT
622 % ------------------------------------------------------------------------
623 dist_total_km = trapz(t, v) / 1000;
624 E_traction_Wh = trapz(t, P_drive - P_regen) / 3600;
625 spec_cons_Wh_km = E_traction_Wh / max(dist_total_km, 0.001); % Energy per km
626
627 % Additional driving range gained by reducing active chiller draw
628 range_gain_pred_km = (E_base_Wh - E_pred_Wh) / spec_cons_Wh_km;
629 range_gain_mpc_km  = (E_base_Wh - E_mpc_Wh) / spec_cons_Wh_km;
630
631 % ------------------------------------------------------------------------
632 % 6. PRINT STEP 6 BENCHMARK COMPARISON TABLE
633 % ------------------------------------------------------------------------
634 fprintf↙
('\n========================================================================↙
====================\n');
635 fprintf('              STEP 6: COMPREHENSIVE THERMAL CONTROLLER↙
BENCHMARK                \n');
636 fprintf↙
('========================================================================↙
==================\n');
637 fprintf('%-30s | %-10s | %-12s | %-12s | %-10s\n', 'Metric', 'Uncooled', 'Step 4 Base', 'Step↙
5 Pred', 'Step 5 MPC');
638 fprintf('-------------------------------------------------------------------------------------\n');
639 fprintf('%-30s | %-10.2f | %-12.2f | %-12.2f | %-10.2f\n', 'Peak Temperature (°C)',↙
T_peak_uncooled, T_peak_base, T_peak_pred, T_peak_mpc);
640 fprintf('%-30s | %-10.0f | %-12.0f | %-12.0f | %-10.0f\n', 'Time > 35°C Limit (s)',↙
time_above_35_uncooled, time_above_35_base, time_above_35_pred, time_above_35_mpc);
641 fprintf('%-30s | %-10.0f | %-12.0f | %-12.0f | %-10.0f\n', 'Time > 32°C Setpoint (s)', sum↙
(T_batt > 32.0)*dt, time_above_32_base, time_above_32_pred, time_above_32_mpc);
642 fprintf('%-30s | %-10s | %-12.2f | %-12.2f | %-10.2f\n', 'Chiller Energy (Wh)', 'N/A',↙
E_base_Wh, E_pred_Wh, E_mpc_Wh);
```

```
643 fprintf('%-30s | %-10s | %-12s | %-12.2f | %-10.2f\n', 'Energy Savings vs Base (%)', 'N/A',↙
'Baseline', savings_pred_pct, savings_mpc_pct);
644 fprintf('%-30s | %-10s | %-12.2f | %-12.2f | %-10.2f\n', 'Actuator Chatter (W/s)', 'N/A',↙
wear_base, wear_pred, wear_mpc);
645 fprintf('%-30s | %-10s | %-12s | %-12.2f | %-10.2f\n', 'Battery Aging Reduction (%)',↙
'N/A', 'Baseline', life_ext_pred_pct, life_ext_mpc_pct);
646 fprintf('%-30s | %-10s | %-12.3f | %-12.3f | %-10.3f\n', 'Added Range Gained (km)', 'N/A',↙
0.0, range_gain_pred_km, range_gain_mpc_km);
647 fprintf↙
('==========================================================================↙
==================\n\n');
648
649 % ------------------------------------------------------------------------
650 % 7. STEP 6 DIAGNOSTIC VISUALIZATIONS
651 % ------------------------------------------------------------------------
652 figure('Name', 'Step 6: Comprehensive Controller Benchmarking Analysis', 'NumberTitle',↙
'off');
653
654 % Subplot 1: Temperature Trajectories vs Safety & Setpoint Limits
655 subplot(3,1,1);
656 plot(t, T_batt, 'r--', 'LineWidth', 1.2); hold on;
657 plot(t, T_batt_baseline, 'b:', 'LineWidth', 1.5);
658 plot(t, T_batt_predictive, 'g--', 'LineWidth', 1.5);
659 plot(t, T_batt_MPC, 'm', 'LineWidth', 2.0);
660 yline(35.0, 'k--', 'Safety Threshold (35°C)', 'LineWidth', 1.2);
661 yline(32.0, 'c--', 'Target Setpoint (32°C)', 'LineWidth', 1.2);
662 ylabel('Temp [°C]'); grid on;
663 title('1. Temperature Profile Comparison across Controller Architectures');
664 legend('Uncooled', 'Step 4 Reactive Baseline', 'Step 5 Predictive', 'Step 5 Full MPC',↙
'Location', 'northwest');
665
666 % Subplot 2: Cumulative HVAC Electrical Energy Draw
667 subplot(3,1,2);
668 plot(t, E_cum_base, 'b:', 'LineWidth', 1.5); hold on;
669 plot(t, E_cum_pred, 'g--', 'LineWidth', 1.5);
670 plot(t, E_cum_mpc, 'm', 'LineWidth', 2.0);
671 ylabel('Energy [Wh]'); grid on;
672 title('2. Cumulative Cooling Electrical Energy Draw (Chiller + Pump)');
673 legend('Step 4 Baseline', 'Step 5 Predictive', 'Step 5 Full MPC', 'Location', 'northwest');
674
675 % Subplot 3: Relative Battery Degradation Kinetics k_deg(T)
676 subplot(3,1,3);
677 plot(t, k_deg_base, 'b:', 'LineWidth', 1.5); hold on;
678 plot(t, k_deg_pred, 'g--', 'LineWidth', 1.5);
```

```
679 plot(t, k_deg_mpc, 'm', 'LineWidth', 2.0);
680 ylabel('k_{deg} Rate'); xlabel('Time [s]'); grid on;
681 title('3. Instantaneous Battery Aging Rate (Arrhenius SEI Growth Model)');
682 legend('Step 4 Baseline', 'Step 5 Predictive', 'Step 5 Full MPC', 'Location', 'northwest');
683
684
685 %%↙
==========================================================================↙
=
686 %% PART 6: KALMAN FILTER SOC ESTIMATOR
687 %%↙
==========================================================================↙
=
688 noise = randn(N, 1) * 0.5;
689 V_meas = V_batt + noise;
690 SOC_est = zeros(N, 1);
691 SOC_est(1) = SOC_init;
692
693 P_cov = 1e-4;   % Estimation covariance
694 Q_proc = 1e-6;  % Process noise
695 R_meas = 0.5;   % Measurement noise
696
697 for k = 2:N
698     % SOC prediction step
699     SOC_pred = SOC_est(k-1) - (I_batt(k) * dt) / (Batt_Cap_Ah * 3600);
700     P_cov = P_cov + Q_proc;
701
702     % Measurement prediction
703     V_oc_pred = interp1(SOC_curve, OCV_curve, SOC_pred, 'linear', 'extrap');
704     H = 100; % Sensitivity linear fit factor
705
706     % Kalman gain calculation
707     K = P_cov * H / (H * P_cov * H + R_meas);
708
709     % State update
710     SOC_est(k) = SOC_pred + K * (V_meas(k) - V_oc_pred);
711     SOC_est(k) = max(0.05, min(1.0, SOC_est(k)));
712     P_cov = (1 - K * H) * P_cov;
713 end
714
715 %%↙
==========================================================================↙
=
716 %% PART 7: PERFORMANCE & ENERGY METRICS PRINT OUT
```

```
717 %%↙
==========================================================================↙
=
718 dist_km = trapz(t, v) / 1000;
719 E_drive_Wh = trapz(t, P_drive) / 3600;
720 E_regen_Wh = trapz(t, P_regen) / 3600;
721 E_batt_net = E_drive_Wh - E_regen_Wh;
722 T_max = max(T_batt);
723 T_rise = T_max - T_amb(1);
724
725 fprintf('\n============================================================\n');
726 fprintf('        UNIFIED EV THERMAL SIMULATION SUMMARY          \n');
727 fprintf('============================================================\n');
728 fprintf('Total Duration             : %.0f s\n', t(end));
729 fprintf('Total Distance Travelled   : %.3f km\n', dist_km);
730 fprintf('Net Battery Energy Consumed : %.2f Wh\n', E_batt_net);
731 fprintf('Peak Heat Generation       : %.2f W\n', max(Q_heat));
732 fprintf('Peak Ohmic Heat Loss       : %.2f W\n', max(Q_ohmic));
733 fprintf('Maximum Battery Temperature : %.2f °C\n', T_max);
734 fprintf('Temperature Rise (Uncooled) : %.2f °C\n', T_rise);
735 fprintf('Initial SOC                : %.2f %%\n', SOC(1)*100);
736 fprintf('Final SOC                  : %.2f %%\n', SOC(end)*100);
737 fprintf('============================================================\n\n');
738
739 %%↙
==========================================================================↙
=
740 %% PART 8: SYSTEM VISUALIZATION PLOTS (ANNOTATED SCENARIOS)
741 %%↙
==========================================================================↙
=
742 figure('Name', 'Step 3: Multi-Phase Dynamic Load Profiles', 'NumberTitle', 'off');
743
744 % --- Subplot 1: Speed ---
745 subplot(4,1,1);
746 plot(t, v * 3.6, 'b', 'LineWidth', 1.5); hold on;
747 xline(600, 'k--', 'End of Driving');
748 ylabel('Speed [km/h]'); grid on;
749 title('Scenario 1: Vehicle Speed Profile (Aggressive Driving)');
750
751 % --- Subplot 2: Current ---
752 subplot(4,1,2);
753 plot(t, I_batt, 'r', 'LineWidth', 1.5); hold on;
754 xline(700, 'g--', 'Fast Charge Start');
```

```
755 xline(1600, 'g--', 'Fast Charge End');
756 ylabel('Current [A]'); grid on;
757 title('Scenario 1 & 2: Battery Current (Drive/Regen Pulses & 300A Charge Surge)');
758
759 % --- Subplot 3: Heat Rate ---
760 subplot(4,1,3);
761 plot(t, Q_ohmic, 'r--', 'LineWidth', 1.2); hold on;
762 plot(t, Q_entropic, 'g--', 'LineWidth', 1.2);
763 plot(t, Q_heat, 'k', 'LineWidth', 1.5);
764 xline(700, 'b--', 'Scenario 2 Spike');
765 ylabel('Heat Rate [W]'); grid on;
766 legend('Q_{ohmic}', 'Q_{entropic}', 'Q_{total}');
767 title('Heat Generation Decomposition (Ohmic + Entropic Formulation)');
768
769 % --- Subplot 4: Thermal Response ---
770 subplot(4,1,4);
771 plot(t, T_amb, 'm', 'LineWidth', 1.5); hold on;
772 plot(t, T_batt, 'c', 'LineWidth', 1.5);
773 yline(35, 'r--', 'Thermal Safety Limit (35°C)');
774 xline(700, 'k--', 'Scenario 2 Heatup');
775 xlabel('Time [s]'); ylabel('Temp [°C]'); grid on;
776 legend('T_{ambient}', 'T_{batt} (Uncooled)');
777 title('Scenario 3: Dynamic Ambient & Uncooled Pack Thermal Response');
778
779 figure('Name', 'Electrical & State of Charge Tracking', 'NumberTitle', 'off');
780 subplot(2,1,1);
781 plot(t, V_batt, 'b', 'LineWidth', 1.5); grid on;
782 ylabel('Terminal Voltage [V]');
783 title('Battery Voltage Behavior Under Dynamic Loads');
784
785 subplot(2,1,2);
786 plot(t, SOC * 100, 'k', 'LineWidth', 2); hold on;
787 plot(t, SOC_est * 100, 'r--', 'LineWidth', 1.5);
788 ylabel('SOC [%]'); xlabel('Time [s]'); grid on;
789 legend('True SOC', 'Kalman Estimated SOC');
790 title('State of Charge Estimation Tracking');
791
792 %%↙
═══════════════════════════════════════════════════════════════════════↙
═
793 %% PART 9: EXPORT SIGNALS FOR SIMULINK / SIMSCAPE PLANT
794 %%↙
═══════════════════════════════════════════════════════════════════════↙
═
```

```
795 Q_heat_ts = timeseries(Q_heat, t);
796 Q_heat_ts.Name = 'Q_heat_ts';
797 v_ts = timeseries(v, t);
798 v_ts.Name = 'v_ts';
799 SOC_ts = timeseries(SOC, t);
800 SOC_ts.Name = 'SOC_ts';
801 I_batt_ts = timeseries(I_batt, t);
802 I_batt_ts.Name = 'I_batt_ts';
803 % Convert Ambient Temperature to Kelvin for Simscape physical block
804 T_amb_ts = timeseries(T_amb + 273.15, t);
805 T_amb_ts.Name = 'T_amb_ts';
806 % Save workspace variables to EV_Thermal_Inputs.mat
807 save('EV_Thermal_Inputs.mat', 'Q_heat_ts', 'v_ts', 'SOC_ts', 'T_amb_ts', 'I_batt_ts');
808 fprintf('[SUCCESS] All time-series objects exported to Base Workspace &↙
EV_Thermal_Inputs.mat!\n');
809
810 %%↙
==========================================================================↙
=
811 %% MPC OBJECTIVE & STRICT NONLINEAR CONSTRAINT FUNCTIONS
812 %% (MUST BE AT THE VERY BOTTOM OF THE SCRIPT FILE)
813 %%↙
==========================================================================↙
=
814 function J = mpc_cost_function(u, T_0, Q_h, T_a, u_prev, C_th, R_th, dt, H_p, T_set, w_T,↙
w_E, w_dU)
815     T_pred = zeros(H_p + 1, 1);
816     T_pred(1) = T_0;
817     J = 0;
818
819     for k = 1:H_p
820         Q_pas = (T_pred(k) - T_a(k)) / R_th;
821         dT = (Q_h(k) - Q_pas - u(k)) / C_th;
822         T_pred(k+1) = T_pred(k) + dT * dt;
823
824         e_T = T_pred(k+1) - T_set;
825
826         if k == 1
827             du = u(k) - u_prev;
828         else
829             du = u(k) - u(k-1);
830         end
831
832         J = J + w_T * (e_T^2) + w_E * (u(k)^2) + w_dU * (du^2);
```

```
833     end
834 end
835
836 function [c, ceq] = mpc_nonlcon(u, T_0, Q_h, T_a, C_th, R_th, dt, H_p, T_max)
837     T_pred = zeros(H_p + 1, 1);
838     T_pred(1) = T_0;
839
840     for k = 1:H_p
841         Q_pas = (T_pred(k) - T_a(k)) / R_th;
842         dT = (Q_h(k) - Q_pas - u(k)) / C_th;
843         T_pred(k+1) = T_pred(k) + dT * dt;
844     end
845
846     % Enforce T_pred <= 35.0 °C strictly at all prediction steps
847     c = T_pred(2:end) - T_max;
848     ceq = [];
849 end
850
851
852
853
854
855 %↙
==========================================================================↙
==
856 % RUNTIME BENCHMARKING
857 %↙
==========================================================================↙
==
858 elapsed_time = toc; % Stop execution timer
859
860 fprintf('\n=====================================================\n');
861 fprintf('  Total Script Execution Time: %.3f seconds\n', elapsed_time);
862 fprintf('=====================================================\n');
```